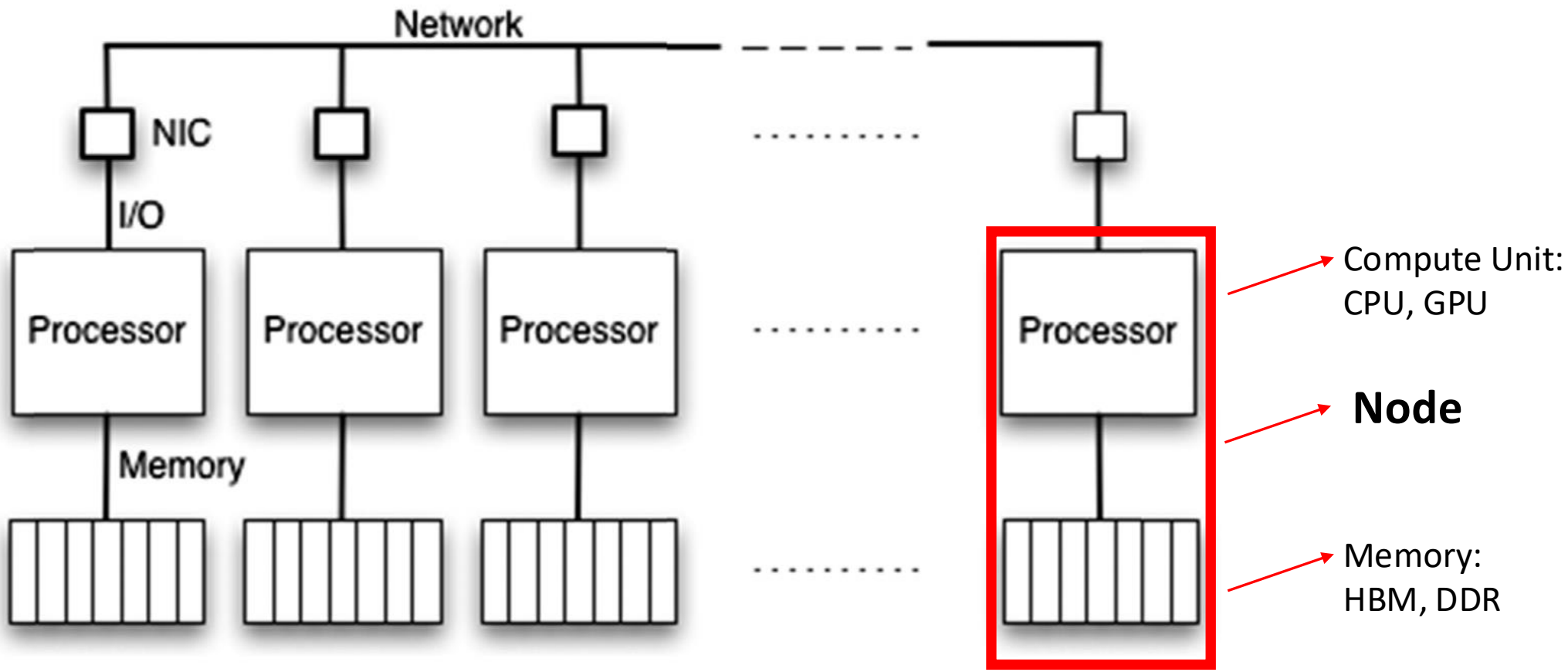
TA Help Session from 2:15

Hardware Topology & NUMA

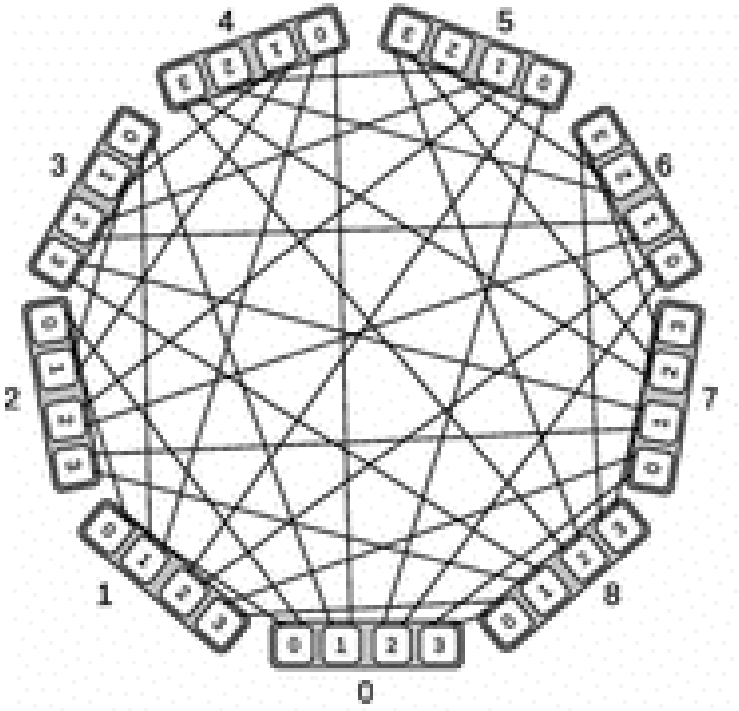
Where we are? – A single Node

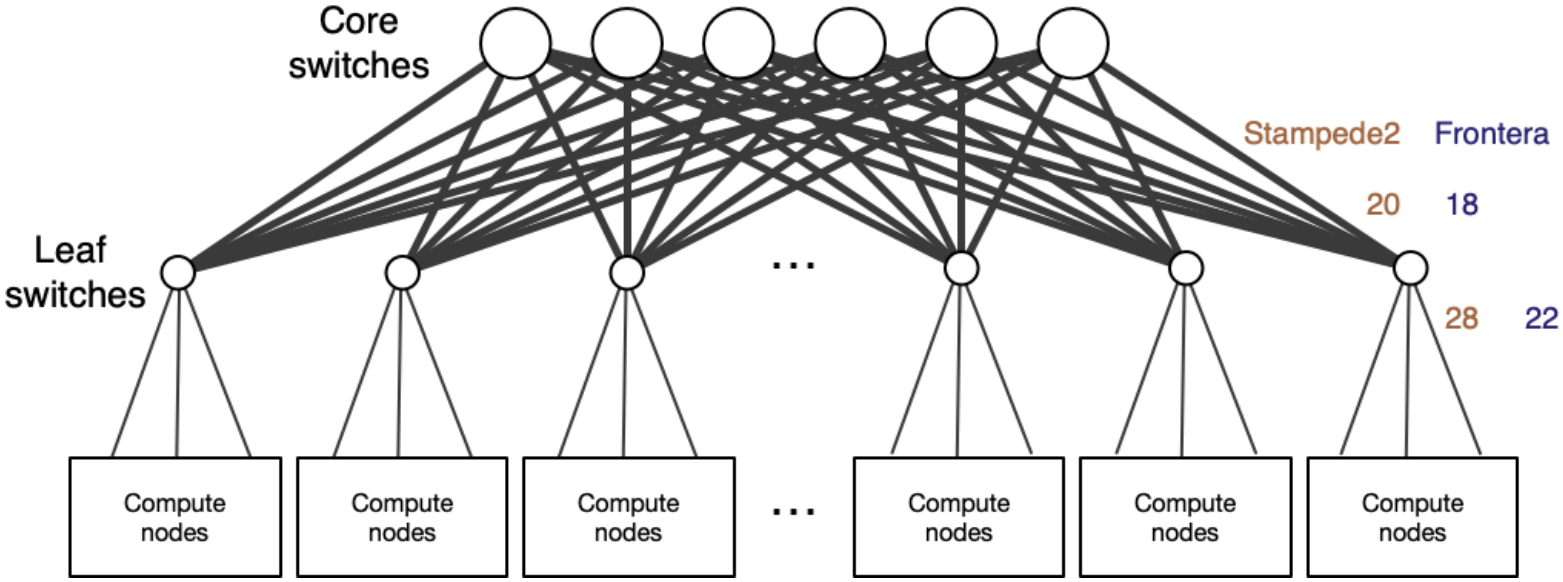


Source: Textbook; High Performance Computing Modern Systems and Practices by Thomas Sterling, Matthew Anderson, and Maciej Brodowic

HPC Network Topology

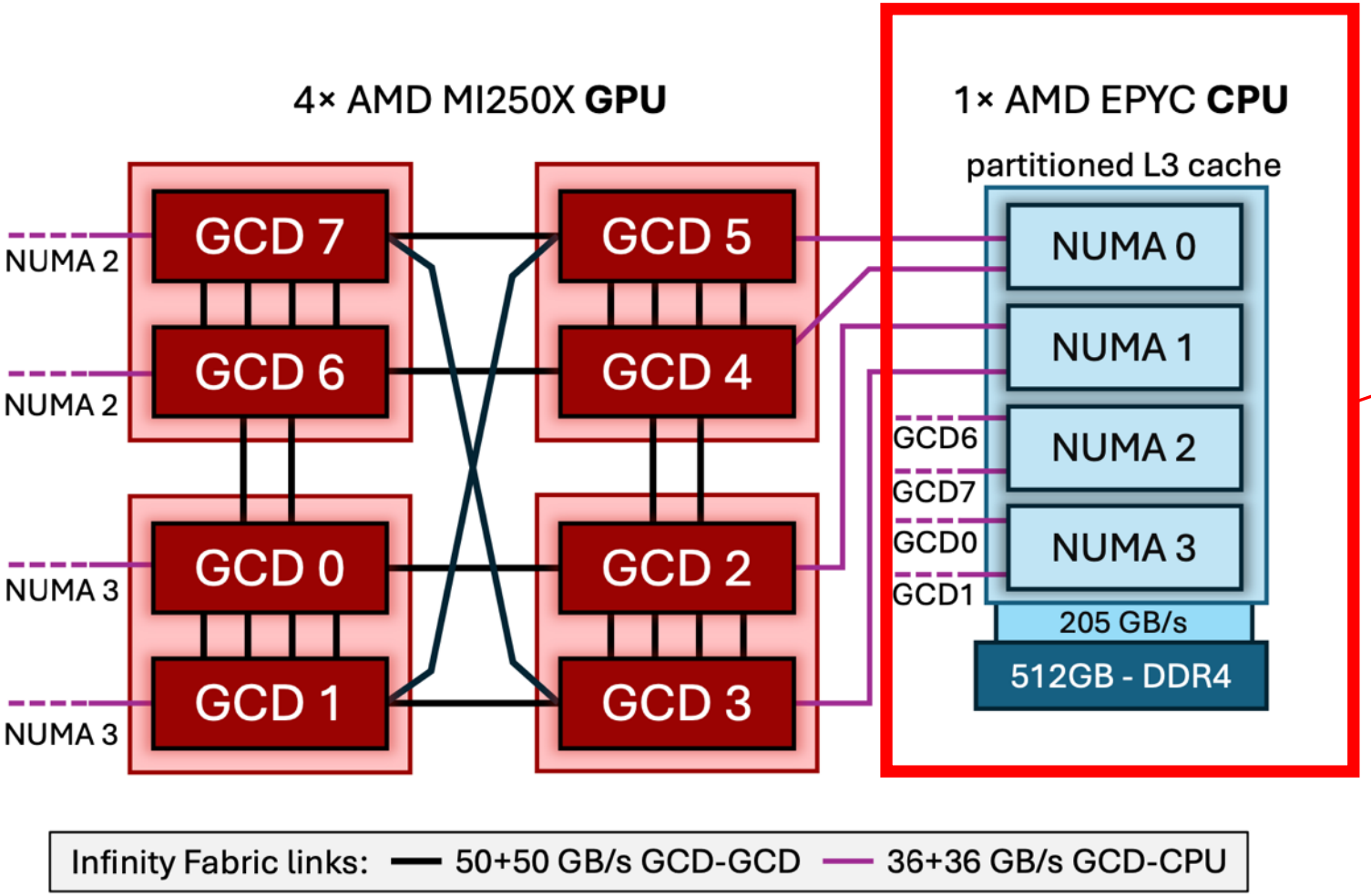
- Two big categories: Fat-tree, Dragonfly





Source: https://cvw.cac.cornell.edu/parallel/hpc/clusters

A Compute Node is Complex



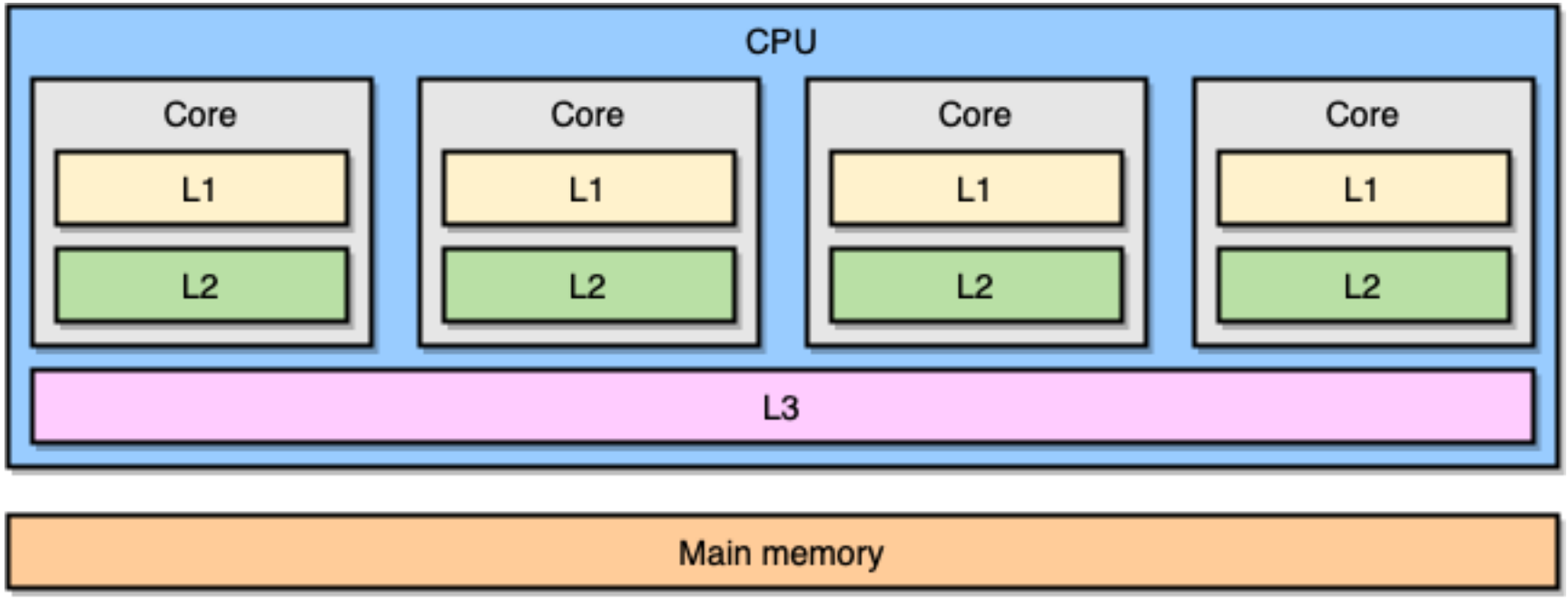
We focus on CPU part for now

Source: Schieffer, Gabin, et al. "Understanding Data Movement in AMD Multi-GPU Systems with Infinity Fabric." *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2024.

Discover Hardware Topology

On a target system, three components need to be discovered for understanding its performance limit:

- CPU cores
- Cache hierarchy
- Main Memory



HWLOC – Hardware Locality

- A C API toolkit for discovering hardware resources in parallel architecture
- Provide a portable abstraction of hierarchical architectures for high-performance computing

- Useful for HPC libraries to develop CPU/Memory locality-aware policy
  - Open MPI library
  - OpenMP runtimes
  - Kokkos C++ portability layer

- We only use it for discovering hardware topology here
  - How many CPU cores
  - How are cache / CPU interconnected
  - How much memory
  - How many **NUMA** nodes

Source: https://hwloc.readthedocs.io/en/stable/index.html#overview

HWLOC – Topology Discovery

- Command line: lstopo

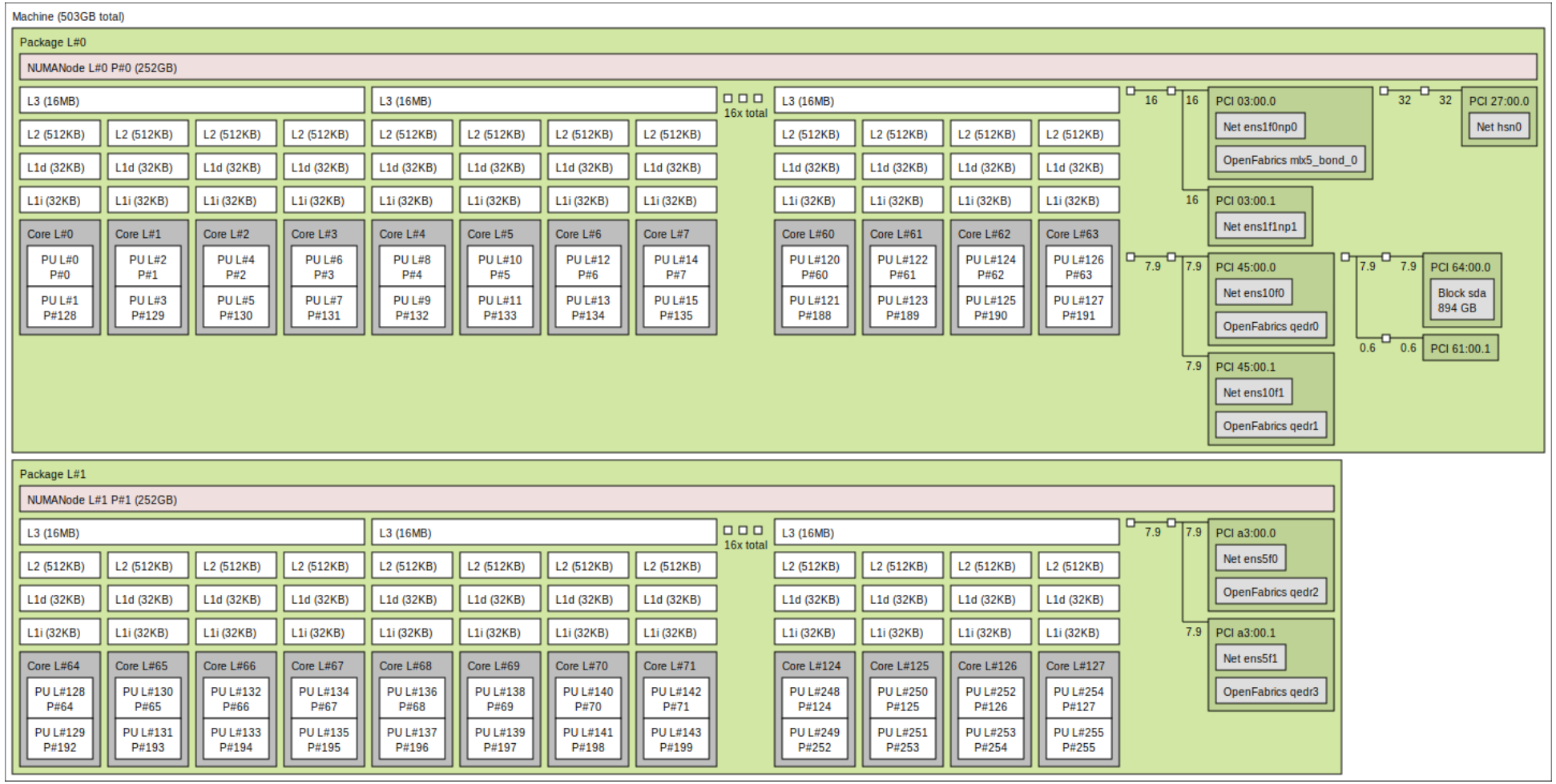
```
Machine (503GB total)
  Package L#0
    NUMANode L#0 (P#0 252GB)
    L3 L#0 (16MB)
      L2 L#0 (512KB) + L1d L#0 (32KB) + L1i L#0 (32KB) + Core L#0
        PU L#0 (P#0)
        PU L#1 (P#128)
      L2 L#1 (512KB) + L1d L#1 (32KB) + L1i L#1 (32KB) + Core L#1
        PU L#2 (P#1)
        PU L#3 (P#129)
      L2 L#2 (512KB) + L1d L#2 (32KB) + L1i L#2 (32KB) + Core L#2
        PU L#4 (P#2)
        PU L#5 (P#130)
      L2 L#3 (512KB) + L1d L#3 (32KB) + L1i L#3 (32KB) + Core L#3
        PU L#6 (P#3)
        PU L#7 (P#131)
    L3 L#1 (16MB)
      L2 L#4 (512KB) + L1d L#4 (32KB) + L1i L#4 (32KB) + Core L#4
        PU L#8 (P#4)
```

...

```
HostBridge
  PCIBridge
    PCI a3:00.0 (Ethernet)
      Net "ens5f0"
      OpenFabrics "qedr2"
    PCI a3:00.1 (Ethernet)
      Net "ens5f1"
      OpenFabrics "qedr3"
```

HWLOC – Topology Discovery

- Visualize the. topology by saving it into a picture *hwloc-ls*



HWLOC – Topology Discovery

- From the previous topology, we can determine:
  - How many cores and CPUs are available?
  - How do they share L1, L2, L3 caches?
  - How many NUMA nodes are there?
  - What is the affinity from a CPU to NUMA node X?

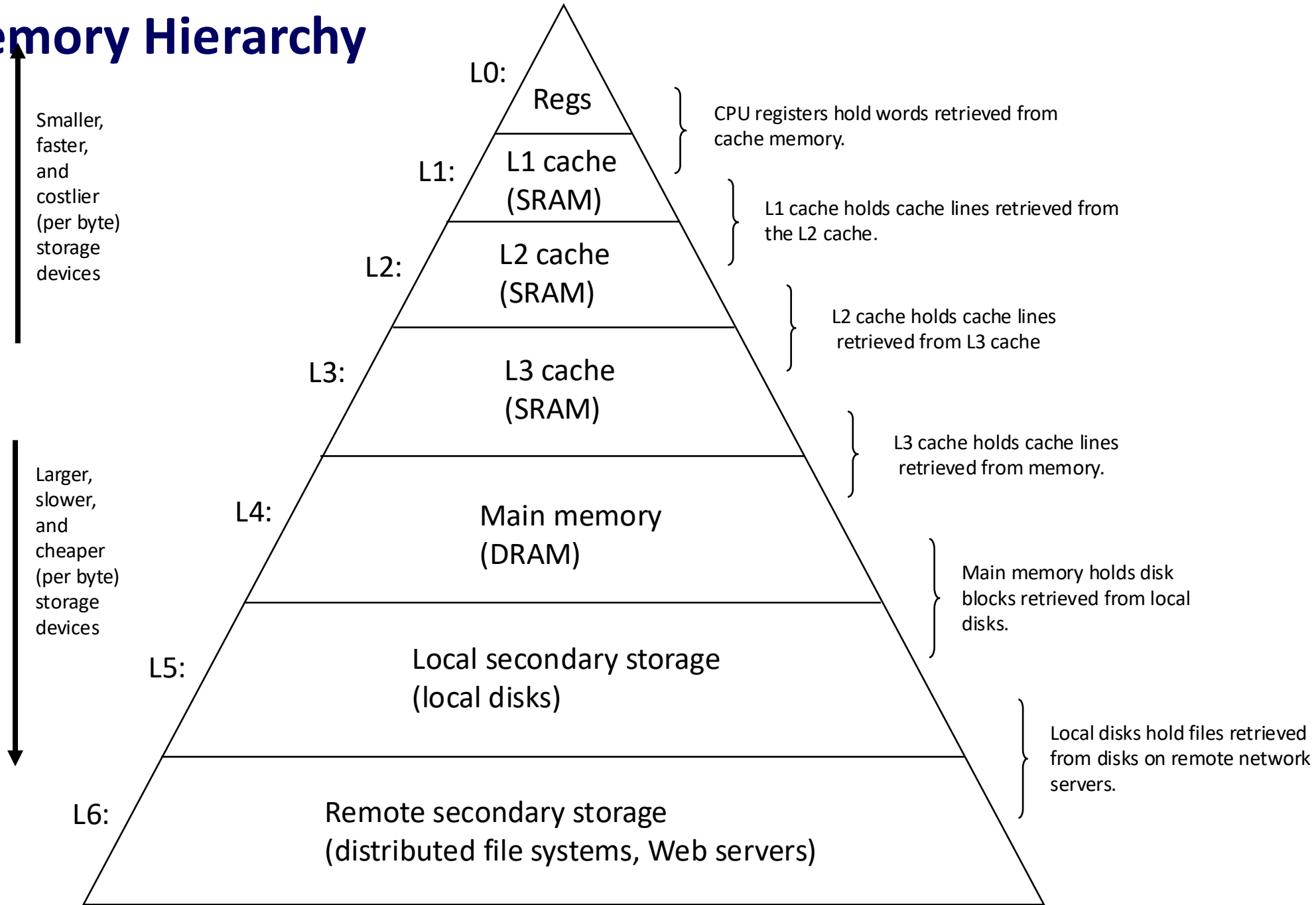
Numactl: an alternative

- A simple interface for discoverying NUMA nodes, available memory, and CPU affinity

```
$:~> numactl –H
```

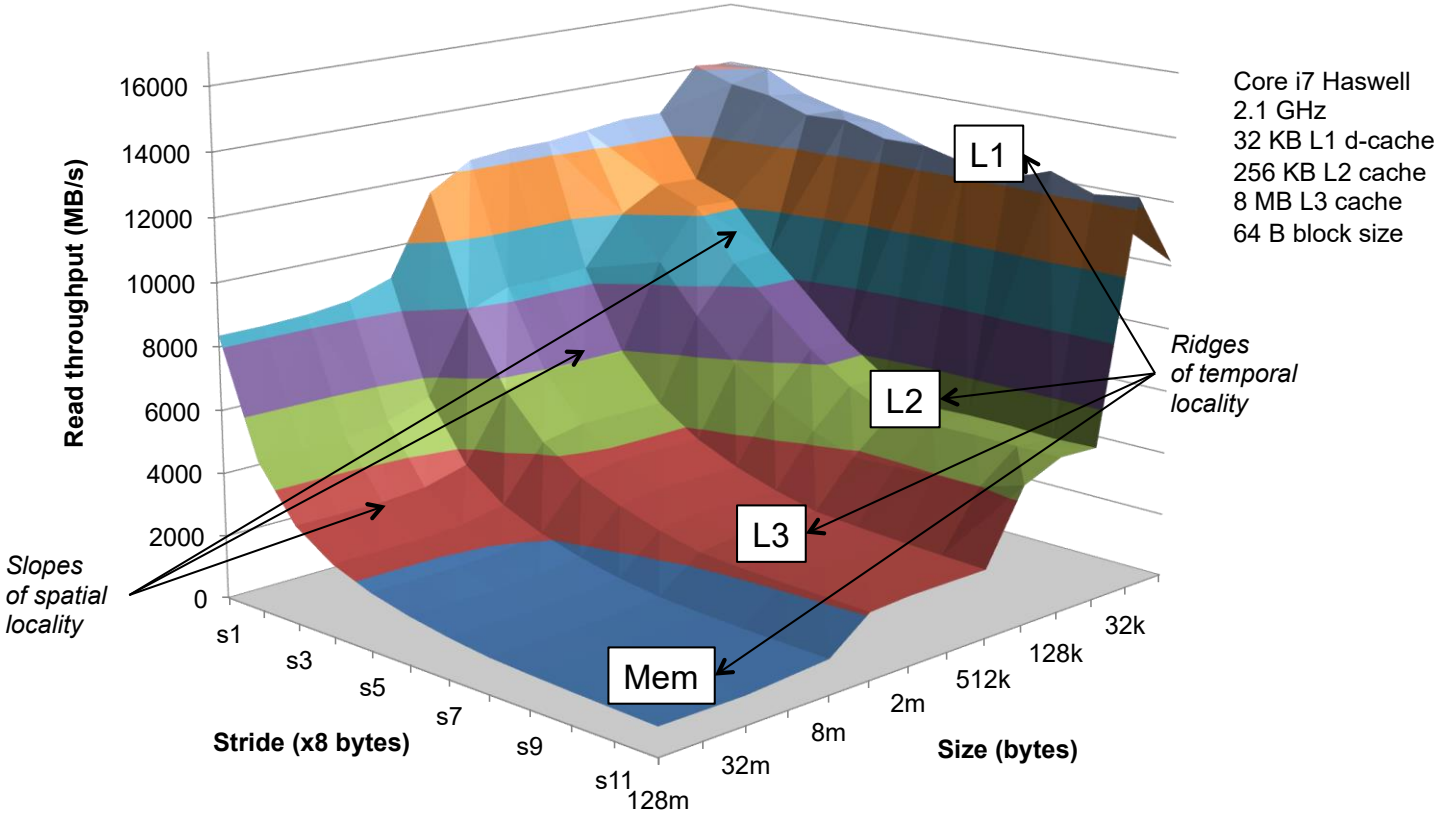
```
available: 2 nodes (0-1)
node 0 cpus: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 2
0 51 52 53 54 55 56 57 58 59 60 61 62 63 128 129 130 131 132 133 134 1
56 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173
node 0 size: 257638 MB
node 0 free: 127791 MB
node 1 cpus: 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82
08 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125
 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 2
49 250 251 252 253 254 255
node 1 size: 257937 MB
node 1 free: 133287 MB
node distances:
node   0   1
  0:  10  32
  1:  32  10
```

Memory Hierarchy



Memory and Cache Bandwidth

- How fast can a core fetch data for its computation?
- A memory mountain example from the book *Computer Systems: A Programmer's Perspective*Links to an external site. by Randal E. Bryant and David R. O'Hallaron.



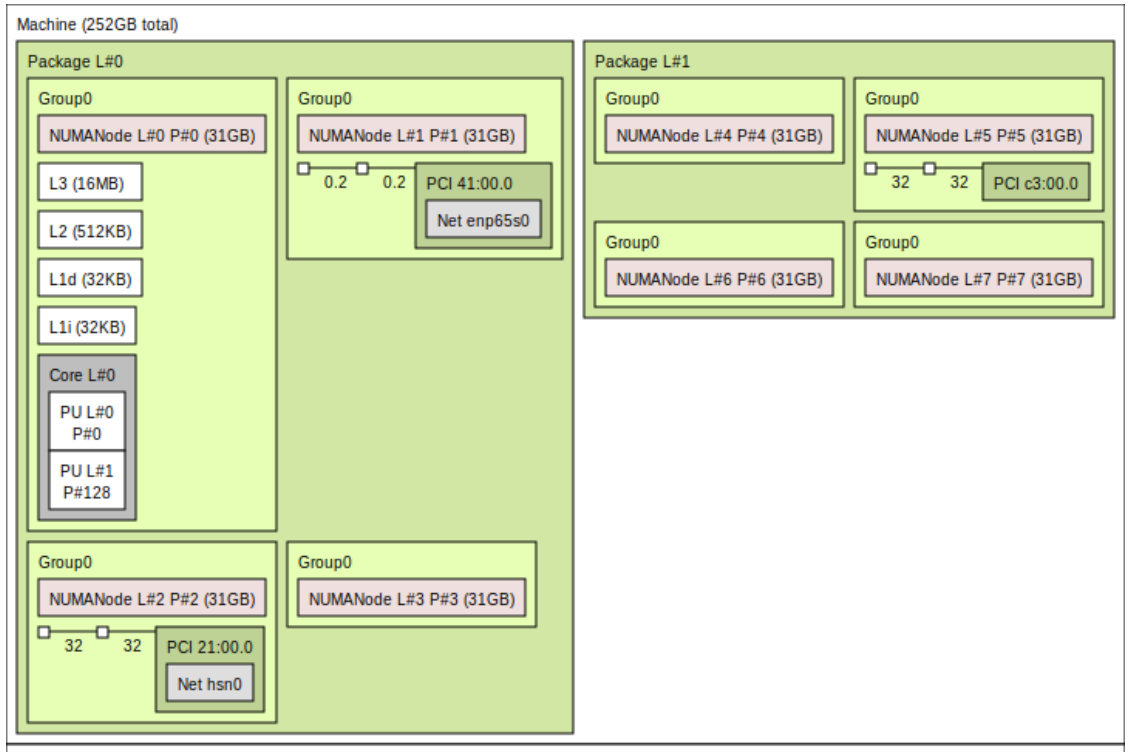
The Stream Benchmark

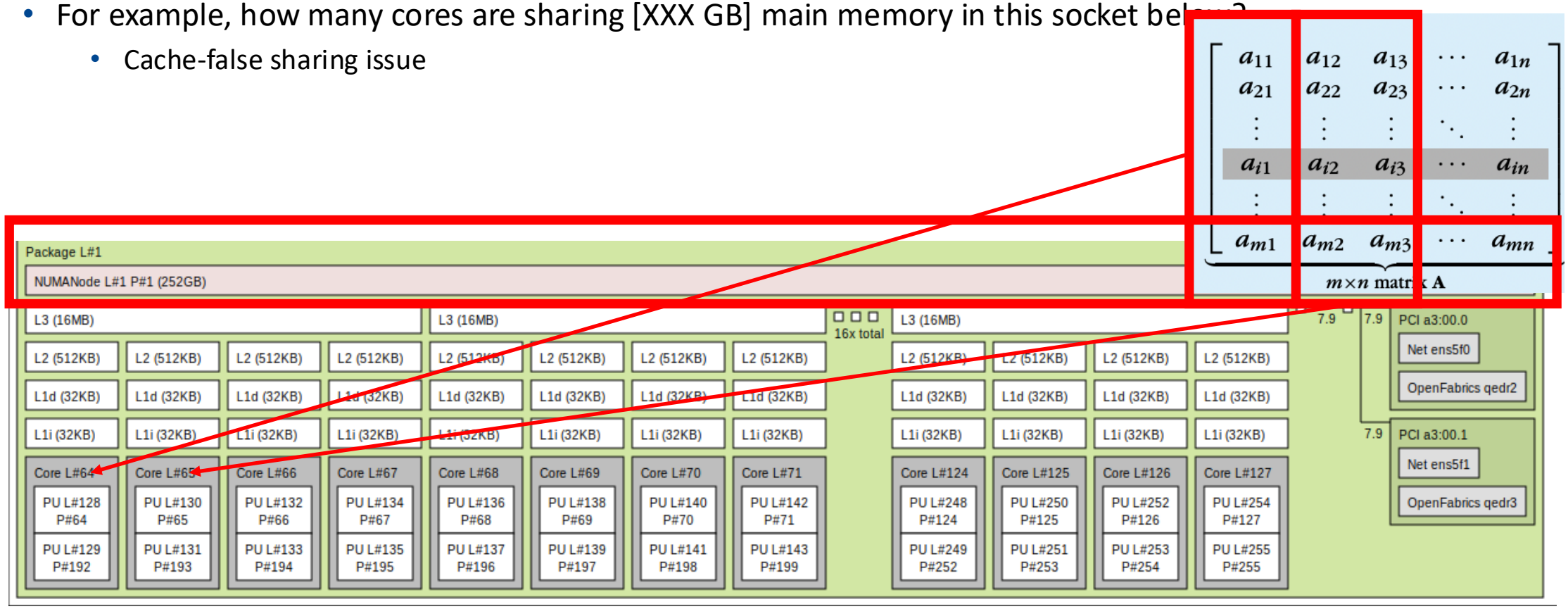
- A widely used benchmark for quantifying ‘sustainable’ memory bandwidth
  - https://www.cs.virginia.edu/stream/
- Consists of 4 kernels:
  - "Copy’’
  - "Scale’’
  - "Sum’’
  - "Triad''
- Need to ensure that the array size is large enough to exceeds last level cache (LLC) capacity:
  - each array must be at least 4x the size of total LLC size
  - Question: how should you set the array size when running on Dardel?
    - Hint: check the :3 on the topology plot

Next Step

- When you run the hwloc and stream on a Dardel node, check how many CPUs/Cores you actually requested in your job allocation. They will impact what topology you will see.

```
$:~> salloc -t 00:10:00 --nodes=1 -A edu26.DD2356 -p shared   --ntasks-per-node=1  --cpus-per-task=2
```



Shared Memory Programming - OpenMP

- In the next module, you will learn about **shared memory** programming model on multi-core processors
- For example, how many cores are sharing [XXX GB] main memory in this socket below?
  - Cache-false sharing issue